Numer Algor
DOI 10.1007/s11075-016-0119-6

ORIGINAL PAPER

Comments on direct transcription solution of DAE constrained optimal control problems with two discretization approaches

Stephen L. Campbell[1] · **John T. Betts**[2]

Received: 9 July 2015 / Accepted: 26 February 2016
© Springer Science+Business Media New York 2016

Abstract There have been a number of results in the literature showing that a direct transcription numerical approach to optimal control problems could have a number of surprising and desirable behaviors that were not always predicted by the existing theory especially when differential algebraic equations are involved. Most of these results were developed with an implicit Runge-Kutta (IRK) discretization being used. It is important to know which of these observations hold for direct transcription software using different types of discretizations and which are discretization specific. This paper reexamines several of these questions but using an hp-pseudospectral code. It is seen that while philosophically the results are often similar to the IRK results, there are some differences that should be understood by users solving constrained optimal control problems. This paper also discusses a type of regularization for higher index problems that does not reduce the index.

Keywords Optimal control · Direct transcription · Pseudospectral · Differential algebraic equation

Research supported in part by NSF Grant DMS-1209251.

✉ Stephen L. Campbell
slc@ncsu.edu

[1] Department of Mathematics, North Carolina State University, Raleigh, NC 27695-8205, USA

[2] Applied Mathematical Analysis, LLC, P.O. Box 1135, Issaquah, WA 98027, USA

Published online: 11 March 2016

1 Introduction

In an optimal control problem there is a dynamical system and a cost. The cost is to be optimized over the allowable controls and possibly also over parameters. There is a very large literature on optimal control problems with ordinary differential equation constraints that we do not cite here. In many applications the dynamical system is a differential algebraic equation or DAE [7]. A DAE is an implicit differential equation. For our purposes we will assume that the DAE is in the form

$$x_1' = f(x_1, x_2, u, t) \tag{1a}$$

$$0 = g(x_1, x_2, u, t), \tag{1b}$$

since that is the form that is accepted by all the direct transcription (DT) codes we are aware of. The functions f, g are assumed smooth in this paper. The variable u is what the user views as the control and x_1, x_2 the state. Sometimes x_1 is called the differential state and x_2 the algebraic state. The constraints (1b) may be part of the model of the process or they may be part of the optimization constraints. Traditionally the number of equations in (1b) is the same as the size of x_2 and to simplify our discussion we will make that assumption although it is not crucial. Unlike much of the literature we do not assume that the Jacobian of g with respect to x_2, denoted g_{x_2}, is always nonsingular. That is, we do not assume the system is index one if u is viewed as an input. Parameters are also often part of the optimization and modeling but we omit them from this discussion.

There is a large theory on DAEs [9, 24, 25] which we do not repeat here. If the optimization is to be performed by control parameterization, the allowable controls are parameterized by a finite number of parameters. Then (1) is integrated numerically treating the control u as known and the cost evaluated. Then the control parameters are used by an optimizer to optimize the cost. In this case the usual DAE theory has to hold.

However, with DT the dynamics and costs and constraints are all discretized and the resulting large sparse nonlinear programming problem (NLP) is solved numerically [2]. The software evaluates the solutions, refines the grids, and resolves until some termination criteria is met. DT is sometimes called simultaneous optimization [22] and is widely used in industry [8].

In the previous work on DT in [3–5, 10, 13, 14, 23] all of the supporting examples were worked with either SOCS or SOCX which are IRK based optimal control codes. It was seen that the usual DAE theory had to sometimes be reinterpreted when using DT.

In this paper we will first briefly describe GPOPS II, SOCX, and common and distinct features in Sections 1.1, 1.2, and 1.3. Sections 2 and 3 look at many of the previous examples from the literature, but now using GPOPS II which has a very different discretization philosophy than SOCX. Section 4 looks at using GPOPS II on some problems with high index DAEs. A way to regularize some higher index systems is given. Then in Section 5.2 the behavior of a new higher order version of SOCX on DAEs is examined. Some concluding comments are made in Section 6.

1.1 GPOPS II

GPOPS II is a DT code that uses an hp-pseudospectral method [12, 28]. There have been a number of papers that have addressed pseudospectral methods and DAEs. We note [15–18, 29, 31]. There have also been comments in papers about the behavior of pseudospectral methods on DAEs, but these comments were often made just in passing. Our intention here is not a theoretical analysis but an examination of actual codes that are available. We are interested in what happens to a user if they just try to solve a problem directly. We do not consider index reduction although reformulation can be a good thing to do on a specific problem. Many of the previous papers exploited index reduction in their work [18, 31]. This paper is a mixture of review, new observations, and computational studies. A careful explanation of GPOPS II can be found in [27]. We just mention a few key aspects here.

GPOPS II is an hp (or ph) pseudospectral DT method. In both the hp and the ph mesh refinement methods the number of mesh intervals, width of each mesh interval, and the degree of the approximating polynomial can be varied, That means that GPOPS II breaks the domain into subintervals and on each subinterval there is a collocation polynomial. When doing grid refinement it can both subdivide into more intervals (the h), or increase the order of the polynomial on a given interval (the p). The difference between hp and ph lies in which is the preferred option so they have different grid refinement logic. The user chooses which.

GPOPS II uses Legendre-Gauss-Radau collocation. The polynomials are taken as Legendre polynomials. With Legendre polynomials greater order of convergence is gotten for a given number of collocation points by making choices other than uniform distribution of the collocation points. In the actual formulation each subinterval is mapped to $[-1, 1]$ for defining the different terms. We do not need that detail but do note that in the general theory of collocation points, there are four classes of collocation points in the interval t_k, t_{k+1}. There are those that include t_k as a collocation point but not t_{k+1}, those that include t_{k+1} but not t_k, those that include both endpoints, and those that include neither endpoint. Which type of collocation is used can impact on how the software responds when control discontinuities are present as happens in some optimal control problems.

With all GPOPS II calculations we used the NLP solver IPOPT which is an interior point method [30]. We used the mesh refinement strategy "hp1" which puts a premium on changing the interval length h. The other settings we used were "sparseCD" for derivatives, "second" level derivatives and the method of "RPMintegration". The initial grid was 10 equally spaced grid points with fourth order polynomials on each grid. Unless stated otherwise, the maximum polynomial order was 10. Generally the answer was found within 10 to 15 iterations or the program ran for a long time and was stopped. Initial guesses were linear interpolation between initial and final values. Choice of initial guess did not have much impact on the linear problems studied here although it can be very significant in solving some complex nonlinear problems by DT [6].

Default error tolerance was either 10^{-6} or 10^{-8}. For some of the experiments with higher index DAEs this had to be relaxed a couple of additional orders less.

1.2 SOCX

SOCX uses implicit Runge-Kutta (IRK) methods for the discretization. The discretizations are the trapezoid (TR) and the Hermite-Simpson (HS) methods both of which are Lobatto IIIA methods [20]. There are a couple of other discretizations available but the second order TR and the fourth order HS are the two for which the sparse linear algebra is optimized. Both sequential quadratic programming (SQP) and interior point (IP) methods can be used to solve the NLP.

The default is to start with TR for two iterations and then switch to HS since this has been shown to increase the ease of getting feasible solutions on the early coarse grids. The mesh refinement strategy is quite sophisticated. When the mesh gets fine, problem size increases, so it is desirable to avoid unnecessary iterations on fine grids. It is known that optimal control problems often suffer from order reduction along parts of the solution due to stiffness or the presence of state constraints. SOCX estimates the order along the grid. Then it solves the optimization problem of where to best add additional grid points. It does this by adding one point at a time. If the current grid has N points, then up to N points can be added with up to 5 points to a subinterval. Once either of those maximums are met a resolve is done. This has the effect of equally distributing the error along the early coarse grids. Termination is based on several criteria, but a leading one is the estimate of the ODE error in the dynamics. Note that this also includes the resolution of the cost. Optimality is not checked but the user knows that if the user's model is good, then the computed control will deliver the computed optimal performance.

SOCX is available from Applied Mathematical Analysis. One can view the IRK methods of SOCX as using polynomials of fixed order that are collocated on both t_k and t_{k+1} for the trapezoid method and on $t_k, \frac{1}{2}(t_k + t_{k+1}), t_{k+1}$ for the Hermite-Simpson method so that both endpoints of the subinterval are included. In the work of [21] neither endpoint is included with the reason given that this makes it easier to work with problems with discontinuities. The choice for GPOPS II is different from both of these. The left endpoint t_k is a collocation point but t_{k+1} is not. We shall see that while this does not seem to make much difference on many smooth problems that there are some differences with state constrained problems where DAE models occur. GPOPS II permits the use of much higher order polynomials than SOCX uses. However, a new version of SOCX is being developed that allows use of a sixth order method similar to TR or HS. This is examined in Section 5.2.

1.3 Common features

Both GPOPS II and SOCX can accept similar formulations but SOCX also can work with delayed systems. We do not consider delayed systems in this paper.

Both SOCX and GPOPS II give the user the choice of using either a sequential quadratic programming (SQP) or an interior point optimizer. It is pointed out in [23] that an active set SQP method may fail LICQ (linear independent constraint qualification) as more constraints get discarded at tighter tolerances. Some examples that

do not satisfy LICQ do satisfy MFCQ (Mangasarian-Fromowitz constraint qualification). There are some positive results on using interior point methods for NLP satisfying MFCQ. In this paper we use GPOPS II with IPOPT which is an open source interior point code [30]. In Section 5.2 we will use SOCX with both an SQP and an interior point method.

Section 2 will look at what is called the virtual index. Section 3 will look at problems where DT provides a type of numerical regularization. Section 4 examines DAEs, especially higher index DAEs. Our discussion is aimed at users of DT software as opposed to developing specific error estimates or convergence results. When we want to stress which discretization is being used we will use DTS to refer to DT using SOCX and DTG to refer to DT using GPOPS II.

Both SOCX and GPOPS II permit the use of phases. That is, problems that are defined differently on different intervals. This is very important for solving many optimal control problems, especially space missions with different stages and different models during different parts of the mission. The use of phases in the context of DAEs raises some interesting questions such as what are the correct continuity boundary relations between the phases if the DAEs have different algebraic and state variables. These issues are not discussed here since that would take us too far afield.

In (1) there are differential variables x_1 and algebraic variables $\{x_2, u\}$. While the user may think of x_2 as an algebraic state and u as a control, within a DT code all algebraic variables are treated the same. In the terminology of SOCX, $\{x_2, u\}$ are algebraic variables. In the terminology of GPOPS II, $\{x_2, u\}$ are all called controls.

When doing the discretization the unknowns are the coefficients of the interpolating polynomials for the differential variables and the values of the algebraic variables on the collocation points.

There is another aspect of using DT that needs to be kept in mind. Consider an equation like

$$x(t) - f(t) = 0. \tag{2}$$

If (2) is part of a typical ODE discretization, then the error in $x(t_k)$ is a combination of the accuracy to which $f(t_k)$ is computed plus machine roundoff. If (2) is inside a function routine, then again the error is evaluation error and roundoff. However, if (2) is just treated as a constraint, then within a DT code (2) is one of probably several hundred, or maybe thousands, of constraints. The accuracy to which the constraints are satisfied is then the error in solving the NLP and is often less than the accuracy to which f is evaluated.

Also the decision about when an inequality constraint like

$$x(t) - f(t) \geq 0 \tag{3}$$

holds is up to the optimizer. Even if the mathematical analysis of an optimal control problem says that the inequality constraint is or is not an equality along the numerically optimum solution, the optimizer may choose to adjust the residual to improve

the overall performance of the NLP. This ability to perturb the numerical solution can be very desirable as shown in [3, 4] and is one of the issues that will be re-examined in this paper in the context of GPOPS II.

It should be noted carefully what our interests are here. Software gets used on a variety of problems once it is in the users' hands and not all users are expert users. We are interested in when the software can be successful even if the problem is not quite the standard problem and in helping the user interpret their solution. Seeking nicer or more numerically robust reformulations can be very useful [11], but that is not what we are investigating here.

2 Virtual index

When using control parameterization, u is considered known and (1) is a DAE in x. However, a different choice of algebraic state and control from the algebraic variables could produce a lower index DAE. The least such index is called the virtual index of the DAE [10, 13, 25]. A key earlier observation was that when working with DT the important thing is the virtual index and not what the user thinks the index is. We reexamine this with GPOPS II. The problem being virtual index one is closely related to the Euler-Lagrange equations being index one [10]. In fact for linear time invariant linear quadratic regulator optimal control problems they are equivalent. See also [1, 26].

A first virtual index problem is Problem 1. If we view this problem as a DAE in x, then it is index two for a given u. However, viewed as a DAE in $\{x_1, u_2\}$ it is index one. Thus with control parameterization we would need an index two DAE integrator where as DT is able to solve this virtual index one problem as an index one problem provided the virtual control is either bounded or appears in the cost in an appropriate nonlinear manner.

Problem 1 Consider the following controlled DAE

$$x_1' = 2x_1 - 3x_2 + 0.5\cos t + u_1 \tag{4a}$$

$$0 = x_1 + u_2 - \sin 4t, \tag{4b}$$

with cost given by

$$\int_0^T q_1 u_1^2 + q_2 u_2^2 + q_3 x_1^2 + q_4 x_2^2 \, dt. \tag{4c}$$

With $q_1 = 2.0$, $q_2 = 0.2$, $q_3 = 2.0$, and $q_4 = 0.1$, we get the solutions shown in Figs. 1 and 2. We see a smooth solution for (1) as expected for an index one problem. DTG was also able to exploit the fact that the virtual index was one.

The next problem from [13] is a more challenging problem where the index in terms of the control is 3 while the virtual index is one. While this problem is linear for a given control it is nonlinear due to the bL and bx_2 terms.

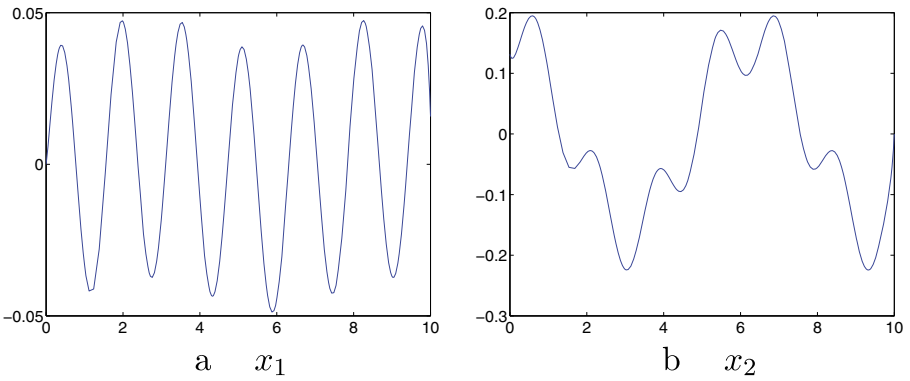


Fig. 1 State solutions for (4) using DTG

Problem 2 (Hockey Puck Problem)

$$\min \quad x_1(1)^2 + x_2(1)^2 + \int_0^1 q_1 L^2 + q_2 b^2 + q_3 c^2 \, dt \tag{5a}$$

$$x_1' = v_1 \tag{5b}$$

$$x_2' = v_2 \tag{5c}$$

$$v_1' = -v_1 + L \tag{5d}$$

$$v_2' = -v_2 - bL \tag{5e}$$

$$0 = x_1 - bx_2 - c \tag{5f}$$

$$-1 = x_1(0), \quad 1 = x_2(0) \tag{5g}$$

$$-2 = v_1(0), \quad 2 = v_2(0). \tag{5h}$$

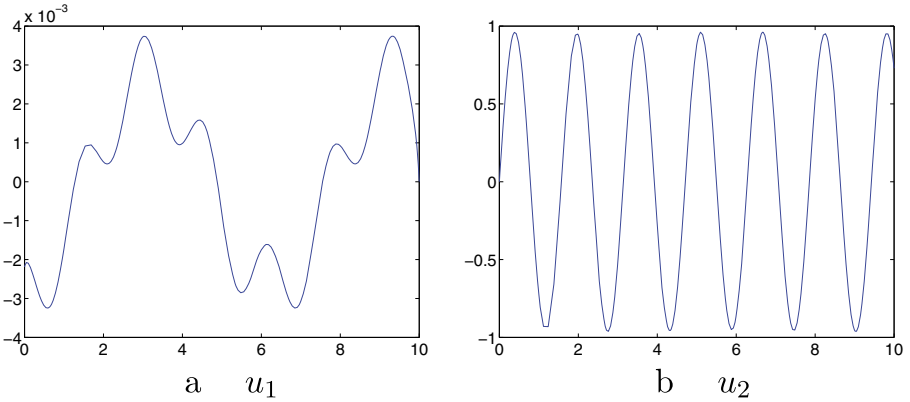


Fig. 2 Controls for (1) using DTG

The dynamics and constraint (5f) can be thought of as a flat surface pushing an object at (x_1, x_2) across a smooth flat surface which has the usual type of friction model proportional to the velocity. The friction coefficient is -1 in (5d), (5e). The control L is an algebraic state variable representing the force exerted by the surface. Controls b, c, are under the user's control and determine the location and slope of the pushing surface. The initial conditions specify that the object starts at point $(-1, 1)$, moving away from the origin with initial velocity $(-2, 2)$. The objective is to push the object close to the origin. The cost term $x_1(1)^2 + x_2(1)^2$ penalizes not ending near the origin. The integrand part of the cost keeps the algebraic variables bounded, or small, depending on the weights q_1, q_2, q_3 provided the weights are nonzero. The positive definiteness of

$$U_2^T \nabla^2_{L,b,c} \theta_0 U_2 = \begin{bmatrix} q_1 & 0 \\ 0 & q_2 + x_2^2 q_3 \end{bmatrix}, \tag{6}$$

where θ_0 is the integrand of the cost (5a), determines whether we have a proper choice of virtual control resulting in an index one virtual control with an finite optimal solution.

The solution for one case when $q_i > 0$ appears in Figs. 3 and 4. What we see from these computations is that if the optimal control problem has virtual index one, then DTG is finding a smooth solution just like DTS did. This is not due to the ability of DTG to integrate higher index DAEs as shown by the other examples later in this paper since there is no chatter in the highest index variables near the right endpoint.

If we set $q_1 = 0$, then DTG failed to converge with our default settings. With $q_1 = 0.000001$, stopping at 15 iterations the controls were extremely noisy and the trajectory poor. For some other values which made (6) singular, MATLAB crashed while using GPOPS II. However, sometimes GPOPS II returned a solution. For example, with $q_1 = 0.01$ and $q_2 = q_3 = 0$ we quickly got an answer with 10 iterations. Setting the number of iterations higher gave essentially same answer. Looking at Fig. 5 we see the state trajectory in Fig. 5a which appears smooth. But looking at the velocity in Fig. 5b we see wiggles. Finally in Fig. 5c we see that the control bounces rapidly. This is consistent with Figs. 7 and 8 in [13] where this same case was solved with SOCX and there were oscillations in the angle of the blade of the hockey stick even though the center of the hockey stick seemed to move smoothly.

3 Regularization

It has been noted with DTS that DT can also serve as a type of regularization in that theoretical difficulties can be ignored if they happen on small enough scale [4]. This was subsequently analyzed in more detail in [22]. In [22] it was shown that this regularization held for additional Runge Kutta discretizations besides those used in SOCX. We examine this now for pseudospectral methods applied to Problem 3.

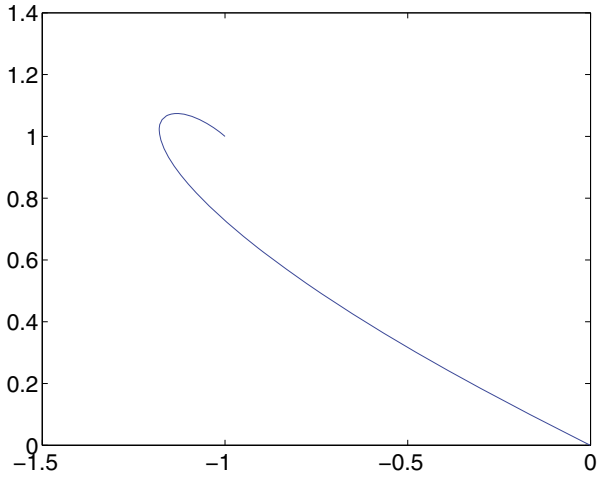


Fig. 3 Hockey puck Problem 2 state trajectory with $q_i = 0.01i$ using DTG

Problem 3 Consider the following roblem with state inequality constraint and touch points. Here $\rho \geq 0$.

$$\overline{J} = \min_{v} \frac{1}{2}\int_0^1 \rho(x_1 - 1)^2 + u^2\, dt \tag{7a}$$

$$x_1' = x_2, \quad x_1(0) = x_1(1) = 0 \tag{7b}$$
$$x_2' = x_3, \quad x_2(0) = 1 = -x_2(1) \tag{7c}$$
$$x_3' = u, \quad x_3(0) = 2 = x_3(1) \tag{7d}$$
$$x_1(t) \leq L. \tag{7e}$$

Note that (7e) is a state inequality constraint. For the choice of $\rho = 0$, problem (3) is the problem studied in [19]. Our problem is motivated in part by the observation

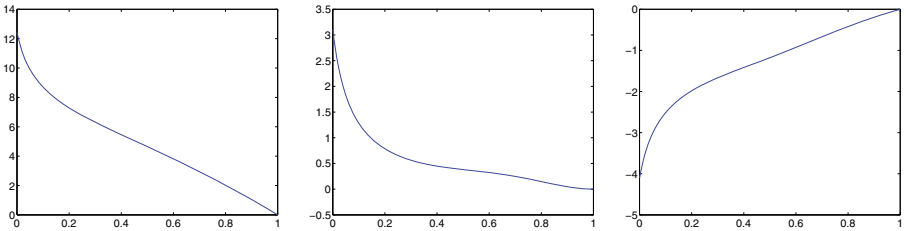


Fig. 4 Controls found for (2) using DTG

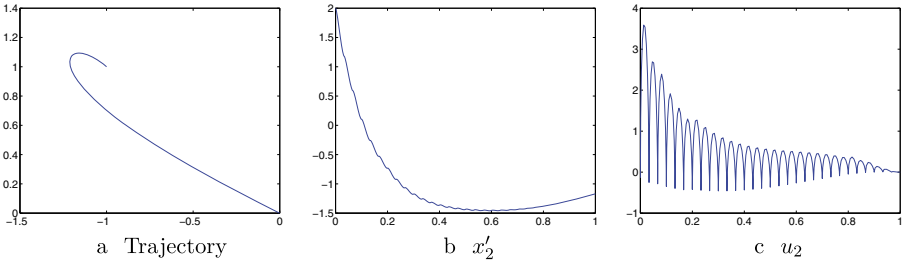


Fig. 5 DTG solution of (2) with $q_1 = 0.01$ and $q_2 = q_3 = 0$

that in a number of applications there can be very different weightings on the control and the state. For example, if we have a control weighting of 10^{-3} and a state weighting of 10^4, which amounts to 10^2 on the norm of x, then the corresponding value of ρ is 10^7. It is known from [19] that the constraint will be active for a range of L values when $\rho = 0$. We take one of these values, $L = 0.134$, throughout this section.

It was shown in [4] that this problem does not have a constraint arc but rather touch points. That is, isolated points where constraint equality holds. However, the movement away from the constraints is very small if ρ is large. DT acts like a regularization as shown with SOXS and SOCS earlier and now with GPOPS II. The solutions are shown in Figs. 6 and 7. Figure 6a appears to show constraint arcs. The touch points are below the level of computational tolerances and thus DT does not need to resolve this very fine detail whereas working from necessary conditions would need to resolve it.

Problem 4 is one that was done in [3] on convergence of non convergent IRK discretizations and also involved an inequality constraint but this one produced a higher index DAE when active (Figs. 8 and 9).

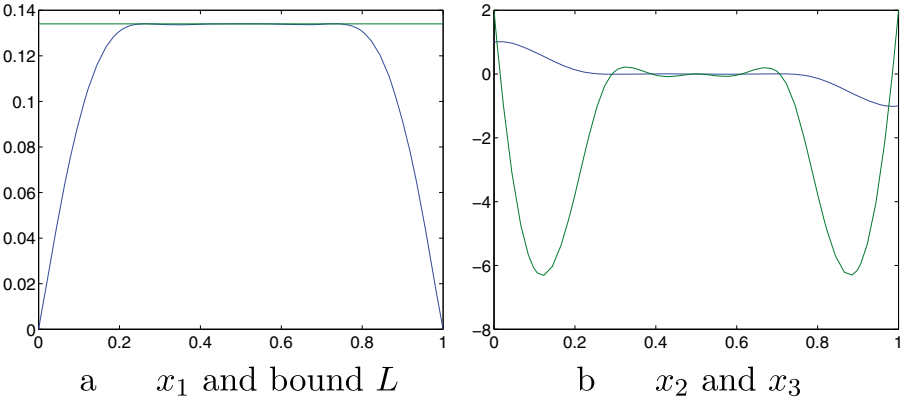


Fig. 6 States for Problem 3 with $\rho = 10000$ using DTG

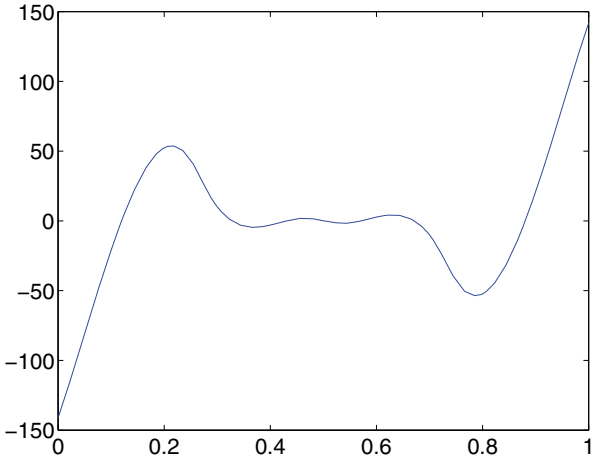


Fig. 7 Control u for Problem 3 using DTG

Problem 4 Problem with active constraint arc.

$$\min \int_0^4 x_1^2 + 10^{-3}u^2 dt \tag{8a}$$

$$x_1' = x_2 \tag{8b}$$

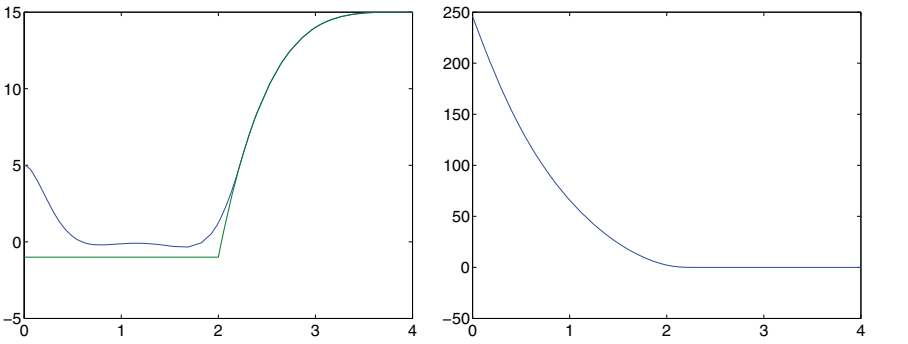
$$x_2' = u \tag{8c}$$

$$0 \leq x_1 - 15 + (t-4)^4 \tag{8d}$$

$$x_1(0) = 5 \tag{8e}$$

$$x_2(0) = 0. \tag{8f}$$

Theoretically the inequality constraint is active on an arc where the theoretical solution is solving an index three problem. This is not a virtual index one problem. With DTS it was seen that this problem was solved by the optimizer slightly



a x_1 and the constraint (8d) b Difference between x_1 and constraint (8d)

Fig. 8 DTG solution of Problem 4

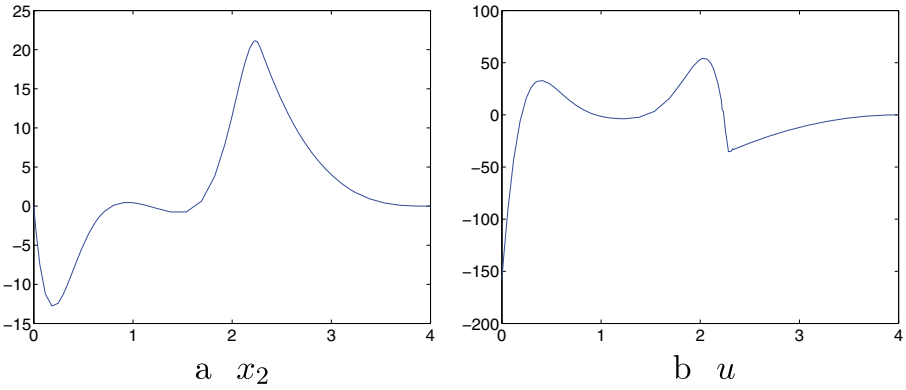


Fig. 9 x_2 and u for Problem 4 using DTG

perturbing the solution off the constraint which it can do since it is an inequality constraint. This small perturbation was exactly what was needed to cancel a large instability due to the IRK discretization not converging on an index three DAE. DTS was not able to integrate the DAE if the constraint arc was required to hold. That is, (9c) holds.

In Problem 5 we check the GPOPS II discretization on a subinterval where constraint equality is to hold.

Problem 5 If we assume equality in (8d) on $[34/35,\ 4]$ we have

$$x_1' = x_2 \tag{9a}$$
$$x_2' = u \tag{9b}$$
$$0 = x_1 - 15 + (t-4)^4. \tag{9c}$$

Problem 5 appears very similar to Problem 7. However, there are some difference. Since the solution is polynomial we have that there is little error since DTG uses high order polynomials.

The calculations are with one iteration. The mesh tolerance was set at 10^{-5} with 4 intervals. The max error in x_1 and x_2 was 10^{-8}. The error in u was 10^{-6} except near the right hand endpoint where it was 10^{-2}. This suggests that the mechanism used by GPOPS II to get an approximate solution of this problem was different than that used by SOCX. GPOPS II could solve Problem 5 and DTS could not, but both could solve Problem 4.

4 DAEs

We now examine how GPOPS II performs with some higher index DAEs. We consider a series of problems with progressively higher index dynamics. In these problems the virtual index is the same as the index and the DT software is being forced to try and integrate a higher index DAE but with a boundary value problem

philosophy rather than an initial value philosophy. Problems 6–8 are the simplest possible higher index problems of index 2, 3, and 4. Success on these problems does not guarantee success on a more difficult problem. However, difficulty on any of these problems indicates difficulty could be expected on a more complex problem of the same index. A more complex linear problem is looked at in Problem 9. A nonlinear problem is discussed in Problem 10.

Problem 6 This is a test of solving an index two DAE.

$$x_1' = x_2 \tag{10a}$$

$$0 = x_1 - \sin(at). \tag{10b}$$

The parameter a is to show the effect of any differentiation. A consistent initial condition is $x_1(0) = 0$ and $x_2(0) = a$. Even though the solution of (10) is unique we take the cost function to be

$$\int_0^T x_1^2 + x_2^2 \, dt. \tag{11}$$

The importance of the cost function will be shown later. The exact solution is $x_1 = \sin(at)$ and $x_2 = a\cos(at)$. Initial conditions are left free when calling the optimizer since they are completely determined by (10).

Solutions for (10) found by GPOPS II are given in Fig. 10.

Figure 10 looks like the correct solution except there is some noise at the right end of x_2 in Fig. 10b. We solved this problem on different length intervals and different requested tolerances and the chatter in x_2 always occurred near the right end of the interval although the chatter was not always the same shape. Working with polynomials and an index two DAE, we expected that we would get x_2 but to reduced order. The chatter at the right endpoint is not what we expected so we will examine this more carefully.

There is some indication that this chatter is due to a free algebraic variable in the NLP problem at the right end of the interval. We again consider (10) but this time we

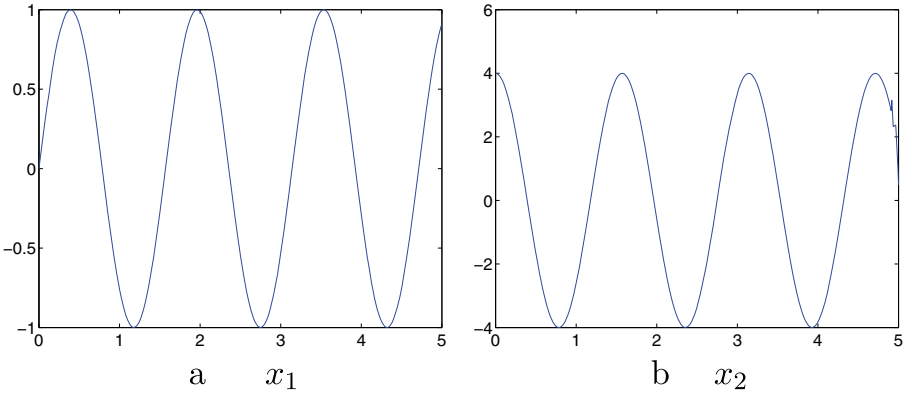


Fig. 10 DTG solution for (10) with $a = 4$, $T = 5$

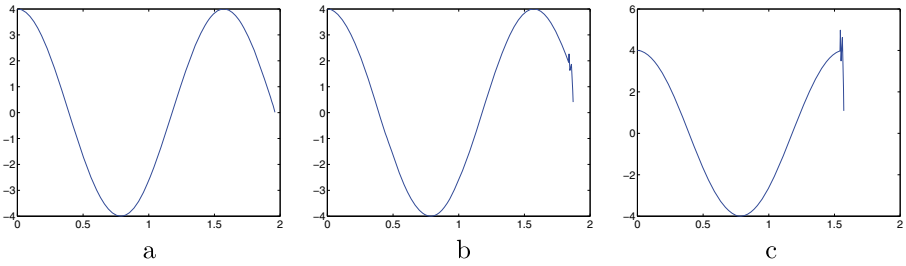


Fig. 11 DTG x_2 solution for (10) with $a = 4$ for different length intervals, $T = 1.9605, 1.8708, 1.5708$

vary the endpoint of the interval so that the algebraic variable of the true solution is at varying distances from zero at the final time. The result is in Fig. 11. We see in Fig. 11a where the algebraic variable ends at zero that there is no apparent chatter. However as we move the endpoint further away from zero we see more and more chatter in Fig. 11b and c. Note also that the final value tries to jump toward zero.

There are two factors here. One is the weighting of x_2 in the cost and the other is the initial guess for x_2 in the call to GPOPS II. In Figs. 12 and 13 we have the initial guess for x_2 is a line from 0 to u_f and we either keep x_2 in the cost or remove it.

Looking at Fig. 12, we see that if the algebraic state x_2 is not in the cost, then the initial guess for $x_2(T)$ appears to be the key factor in determining how much chatter there is in x_2. On the other hand if x_2 is in the cost, then the value of $x_2(T)$ is moved toward the origin independent of the initial guess for $x_2(T)$ as shown in Fig. 13.

We will see a similar endpoint effect in some of the problems that follow. Interestingly it seems to be primarily limited to only the highest index variables which are always algebraic variables. Unless stated otherwise the variable of greatest interest is

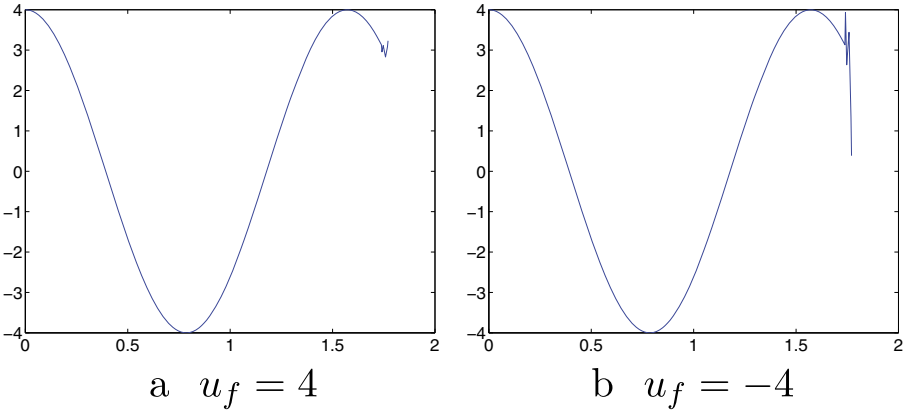


Fig. 12 DTG x_2 solution for (10) without x_2 in the cost (11) with different initial guesses u_f for $x_2(T)$, $T = 1.7708$

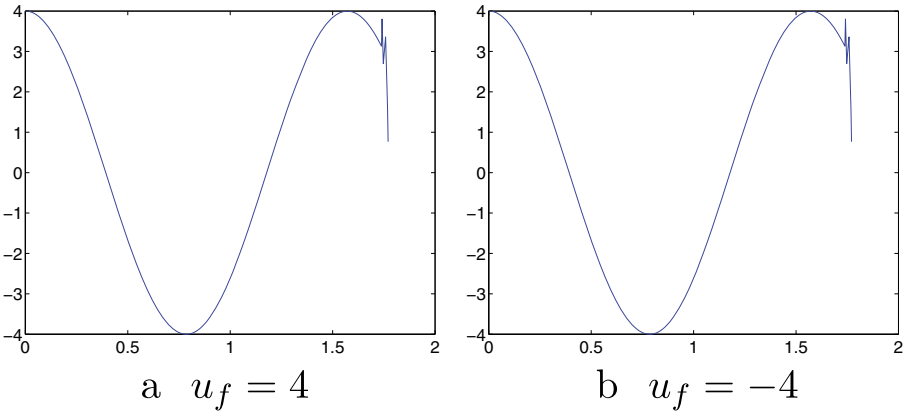


Fig. 13 DTG x_2 solution for (10) with x_2 in the cost (11) with different initial guesses u_f for $x_2(T)$, $T = 1.7708$

the highest index variable so we will focus on it in the remaining discussion in this section.

Consider now what happens if we try to solve an index three problem. Note that the index three problem 7 has the previous problem 6 embedded in it.

Problem 7 This is a test of solving an index three DAE.

$$x_1' = x_2 \tag{12a}$$
$$x_2' = x_3 \tag{12b}$$
$$0 = x_1 - \sin(at). \tag{12c}$$

A consistent initial condition is $x_1(0) = 0$, $x_2(0) = a$, $x_3(0) = 0$. We take the cost function to be $\int_0^T x_1^2 + x_2^2 + x_3^2\, dt$. We do not include the initial conditions in the problem formulation. The exact solution is $x_1 = \sin(at)$, $x_2 = a\cos(at)$, and $x_3 = -a^2 \sin(at)$. With $a = 4$ and making the initial condition free, we get Fig. 14.

This looks like the correct solution except there is some noise at the right end of x_3. We solved this problem on different length intervals and the chatter always occurred near the right side of only x_3. Note that using 10 intervals produced a graph with a lot of corners. This one used $N = 40$. Also we reduced the mesh error from 10^{-8} to 10^{-4} in order to get convergence. The variable x_2 no longer has a jump at the end and appears smooth. Now only x_3 has the noise at the right end.

Figure 15a illustrates how on a different interval we still get the noise at the right end of the graph of the numerical estimate of x_3. Figure 15b shows the error between the computed solution and the exact solution on the right part of the time interval. This computation was with a requested mesh tolerance of 10^{-7}.

Finally, we consider the index 4 Problem 8. Problem 8 contains Problem 7 as a subproblem.

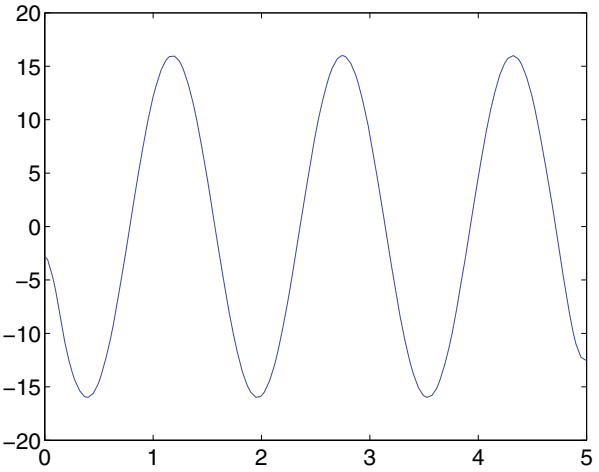


Fig. 14 Index three algebraic variable x_3 for (7) using DTG, $T = 5$

Problem 8 This is a test of solving an index four DAE.

$$x_1' = x_2 \tag{13a}$$
$$x_2' = x_3 \tag{13b}$$
$$x_3' = x_4 \tag{13c}$$
$$0 = x_1 - \sin(at) \tag{13d}$$

with the cost $\int_0^T x_1^2 + x_2^2 + x_3^2 + x_4^2 \, dt$.

With $a = 4$ and making the initial condition free, we get Fig. 16.

Figure 16 looks like the correct solution except there is some large noise at the right end of x_4 and some smaller noise near the peaks of x_4. The noise at the right end of x_4 is the same pattern we saw before in the highest index variable. However, the

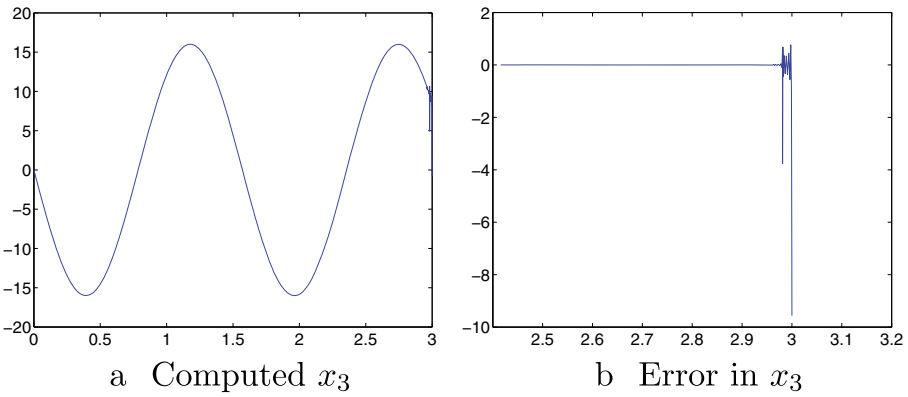


Fig. 15 Index three variable x_3 and its error for (7) using DTG, $T = 3$

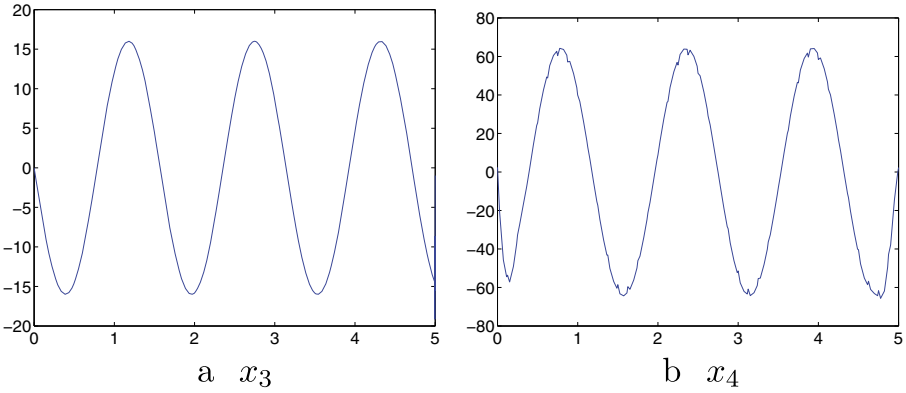


Fig. 16 The index three (x_3) and index four (x_4) variables for (8) using DTG

other noise along the solution is new and may be a reflection of needing to request lower accuracy in order to get reasonable CPU times and avoid memory problems. Note that using 10 intervals produced a graph with a lot of corners. This one used $N = 40$. Also we reduced the mesh error from 10^{-8} to 10^{-3}. The software was running for a very long time even with 10^{-4}.

There is an interesting pattern in the solution of Problems 6, 7, and 8. While we had to use lower tolerances the higher the index, in each case the solutions looked correct except for the highest index variable which had some chatter at the far right of the interval. The chatter at the right of the index two variable in Problem 6 disappears when Problem 6 is a subproblem of Problem 7. The chatter at the right of the index three variable in Problem 7 visually disappears when Problem 7 is a subproblem of Problem 8. Note that as we increase the index, algebraic state variables become differential state variables and thus play a different role in the NLP.

This observation suggests that if we want a smoother approximation to the highest index variable z, we might be able to get it by adding an equation $z' = w$ and then add $\epsilon \|w\|^2$ to the cost. This new problem will have almost the same solution as the original one if ϵ is small, but z is now a differential variable rather than an algebraic variable. Doing this may require accepting a lower order approximation since the index has been raised, but it could in some cases provide useful information on what the solutions look like and thus facilitate modeling and design.

In concluding this section we consider an additional linear index three problem. This example serves several purposes. For one, it is a linear time invariant problem, but there is a manifold of solutions rather than just one. The problem also examines the idea of regularizing in a way that does not reduce the index but is different then the regularization idea of the previous paragraph which raised the index. This problem needs to be semi-explicit since that is required by both GPOPS II and SOCX. We want to know the true solution for evaluation purposes so the problem is devised with a specified solution by using the functions $f_i(t)$. A nonlinear problem is discussed in Problem 10 with a similar structure. So Problem 9 helps to determine which

computational effects are due to the more complex structure and which are due to nonlinearity.

Problem 9 We consider an index three DAE of the form

$$x' = Ax + By + Cz + f_1 + \Theta u \tag{14a}$$
$$y' = Dx + Ey + f_2 \tag{14b}$$
$$0 = Fy + f_3, \tag{14c}$$

where A, B, D, E are 2×2 matrices, F, C^T are 1×2 and FDC is invertible. In particular, we take

$$F = \begin{bmatrix} 0.5 & 0.7 \end{bmatrix}, C = \begin{bmatrix} 0.2 & 0.6 \end{bmatrix}^T, D = I, A = \begin{bmatrix} -1 & 0, 1 \\ 1.1 & -1 \end{bmatrix}, \tag{15}$$

$$B = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix}, E = \begin{bmatrix} -0.5 & 0.2 \\ -0.1 & -1 \end{bmatrix}, \Theta = aI, \tag{16}$$

$$\begin{bmatrix} x_r^T & y_r^T & z_r^T \end{bmatrix} = \begin{bmatrix} \sin t & \cos t & \cos 2.5t & \sin 2.7t & \cos 0.3t \end{bmatrix}, \tag{17}$$

with cost

$$\int_0^T x^T x + y^T y + z^2 + \delta u^T u \, dt. \tag{18}$$

The f_i are defined so that (17) and $u = 0$ are a solution of (9) for appropriate initial conditions determined from (17). The x_i, y_i are the differential states and z is the algebraic state or control in the original formulation. Note that there are additional hidden constraints besides (14c) as to be expected with an index three problem. We take $\Theta = aI$. If $\Theta = 0$, then we just have an index three DAE. If $\Theta \neq 0$ and δ is large relative to Θ, then we have a type of regularization which does not reduce the index.

For this particular study, since we knew an exact solution we took the exact initial conditions as starting values. We used state and control bounds of ± 10. The initial guess for the state was linear interpolation between the end values of (17). The intial guess for the controls was between 1 and $\cos(0.3T)$ for z and ± 0 for u. Bounds on the algebraic state variable z were ± 20 while bounds on the regularizing u were ± 5. Other settings were mesh.tolerance = $1e^{-6}$, mesh.maxiteration = 6, mesh.colpointsmax = 30, mesh.colpointsmin = 4, an initial uniform mesh of 10 points, and $T = 6$. The low number of iterations were set since we expected trouble in meeting the requested tolerances due to the high index nature.

We first solved the problem with $\Theta = 0$. The code terminated after 6 iterations and the asnwer was not any good as shown in Figs. 17 and 18. We see that the error is essentially $O(1)$. Taking more iterations did not improve the situation. Note that x_2, y_2, and z are index one, two, and three variables respectively.

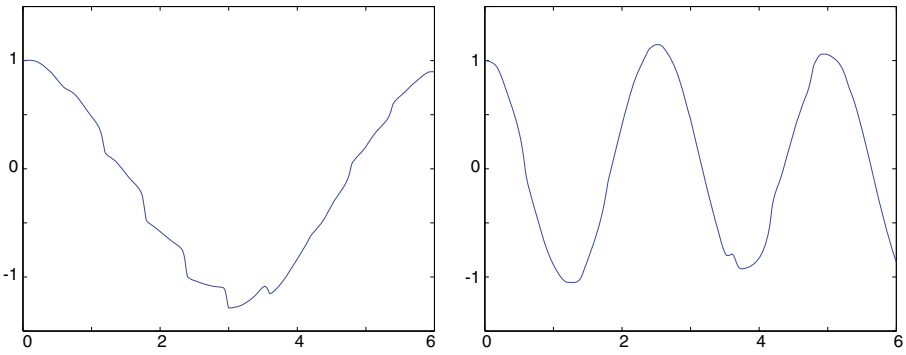


Fig. 17 x_2 (*left*) and y_2 (*right*) for solving Problem 9 with $\Theta = 0$ using DTG

We then solved this same problem with $a = 1$ and $\delta = 10^4$, which is still index three but with the regularizing variables u. Figures 19, 20 and 21 show index one, two, and three solution components and their error when compared to the theoretical solution of the unperturbed problem.

Two things should be noticed. First we are now getting an approximation with error less than 10^{-3}. Secondly as shown in the right side figure of Fig. 21, we again get the spikes near the right end of the interval in the index three variables.

We reran this example with $\delta = 10^6$, allowing 10 iterations, and a requested error tolerance of 10^{-8}. The code ran in just seconds and the graphs were very similar. Error in x was 10^{-4}, but only because of spikes near the right end, error in y was 10^{-6}, error in z again was mainly some spikes near the right end of the interval, but the u was 10^{-6}. The introduction of the additional control u did not reduce the index but did provide a higher quality solution.

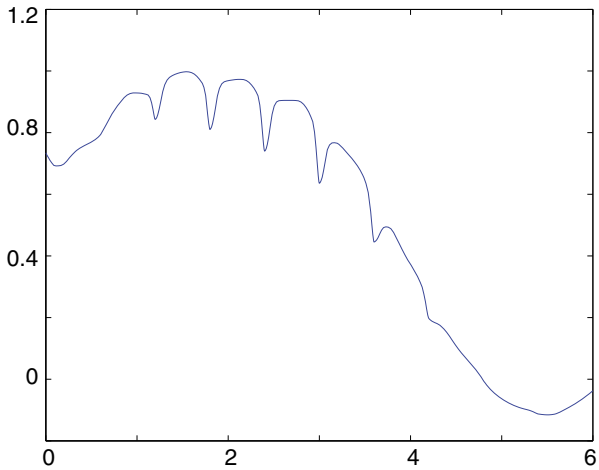


Fig. 18 z for Problem 9 with $\Theta = 0$ when using DTG

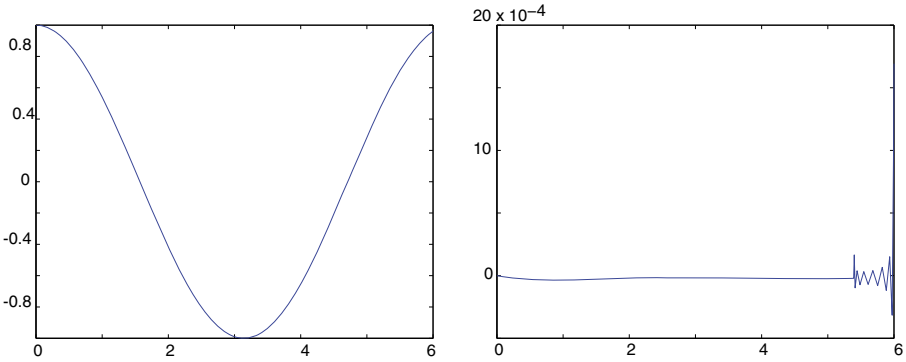


Fig. 19 x_2 (*left*) and error in x_2 (*right*) for Problem 9 with $a = 1$, $\delta = 10^4$ using DTG

5 A nonlinear index three problem

Nonlinearity can have a big impact on performance of DAE optimal control software. In this section we consider a nonlinear index three problem.

5.1 Using DTG for Problem (10)

Using its second and fourth order IRK discretizations SOCX did not perform well on DAEs that are higher index if the DAE was not part of an optimization problem with virtual index one or with inequality constraints. In particular, it did not perform well on higher index DAE boundary value problems (BVPs). In the preceding examples, while there are some reduced order and other issues, the bottom line is that DTG is performing better than DTS on some index two and three DAEs although it cannot solve all such problems. However, GPOPS uses piecewise polynomials of possibly high order. This raises the possibility that this better behavior when it occurs is related to working with linear systems with polynomial or analytic terms.

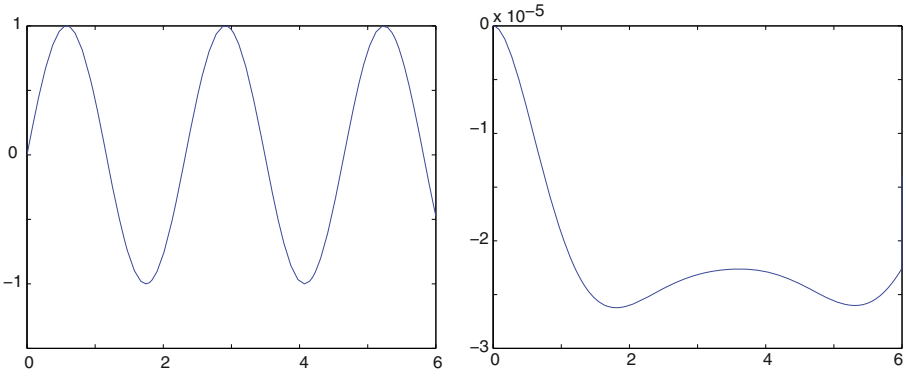


Fig. 20 y_2 (*left*) and error in y_2 (*right*) for Problem 9 with $\Theta = I$, $\delta = 10^4$ using DTG

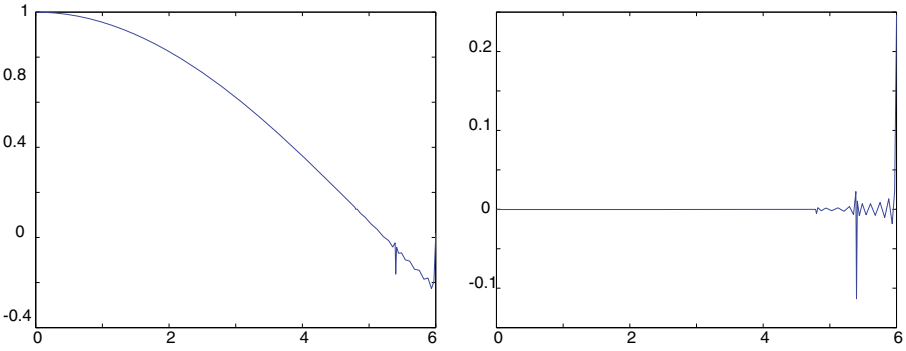


Fig. 21 z (*left*) and error in z (*right*) for Problem 9 with $\Theta = I$, $\delta = 10^4$ using DTG

To investigate this behavior we now consider an index three nonlinear DAE. The example we choose is the classical index three nonlinear pendulum. We consider a variant where there may be an additional force acting in the x direction indicated by u in (20a).

Problem 10 Controlled pendulum problem. We consider the following model for a frictionless pendulum. It is slightly simplified in terms of parameters. The length is L. g is a constant for the force of gravity. u is a force acting on the mass in a horizontal direction. λ is a highly nonlinear function of x, y, x', y', u and represents the force exerted by the path constraint (19c).

$$x'' = -\lambda x + \alpha u \tag{19a}$$
$$y'' = -\lambda y - g \tag{19b}$$
$$0 = x^2 + y^2 - L^2, \tag{19c}$$

or in GPOPS notation,

$$x_1' = x_2 \tag{20a}$$
$$x_2' = -u_1 x_1 + \alpha u_2 \tag{20b}$$
$$x_3' = x_4 \tag{20c}$$
$$x_4' = -u_1 x_3 - g \tag{20d}$$
$$0 = x_1^2 + x_3^2 - L^2. \tag{20e}$$

We take $g = 16$, $L = \sqrt{8}$. Here u_2 is u and u_1 is λ in (20). We consider two variants. One where u is there and one where u is absent. The cost function is taken to be

$$\int_0^T x_1^2 + x_2^2 + x_2^2 + x_4^2 + u_1^2 + \delta\, u_2^2\, dt. \tag{21}$$

The initial conditions are $[x_1(0), x_2(0), x_3(0), x_4(0)] = [2, 0, 2, 0]$.

The parameters α, δ allow us to control whether the dynamics are just an index three DAE when $\alpha = 0$ or a DAE with virtual index three when $\alpha \neq 0$.

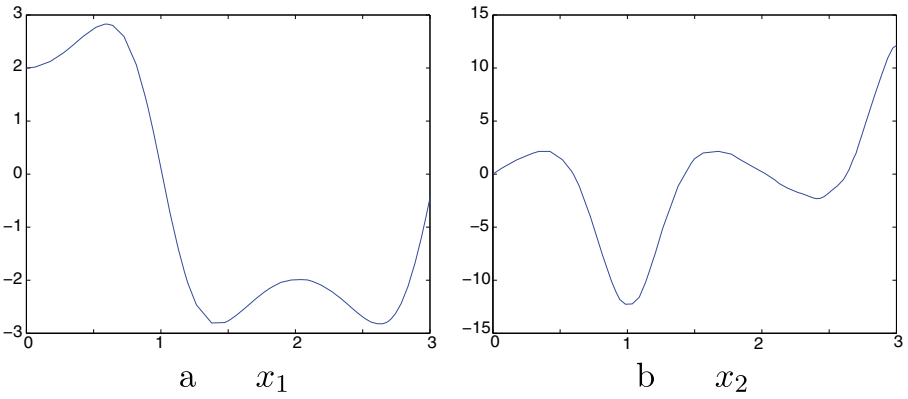


Fig. 22 Pendulum position variables ($\alpha = 0$)

Note that if we differentiate the constraint (20e) twice we get the hidden constraint

$$x_2^2 + x_4^2 - u_1 L^2 - x_3 g + \alpha x_1 u_2 = 0. \tag{22}$$

If $\alpha = 0$, then u_2 plays no role, u_1 is given in terms of the x_i, and the problem is an index 3 DAE with a two dimensional solution manifold. However, if $\alpha \neq 0$ the index 3 DAE still has a one dimensional control input.

Suppose that $\alpha = 0$ and $\delta \neq 0$. Then two things happen. One is that if we try to set the tolerances too high or with two many allowed iterations or a somewhat longer time interval, the software runs for a long time and eventually stops with a full memory error. If we increase the mesh error to 10^{-3} and limit the number of iterations and take the interval to be only $[0, 3]$ we get the solutions shown in Figs. 22, 23 and 24.

We see here the expected behavior in the low index variables. As shown in Fig. 24b the pendulum starts at rest and swings to the left to the same height and then starts back. This is the expected pendulum path.

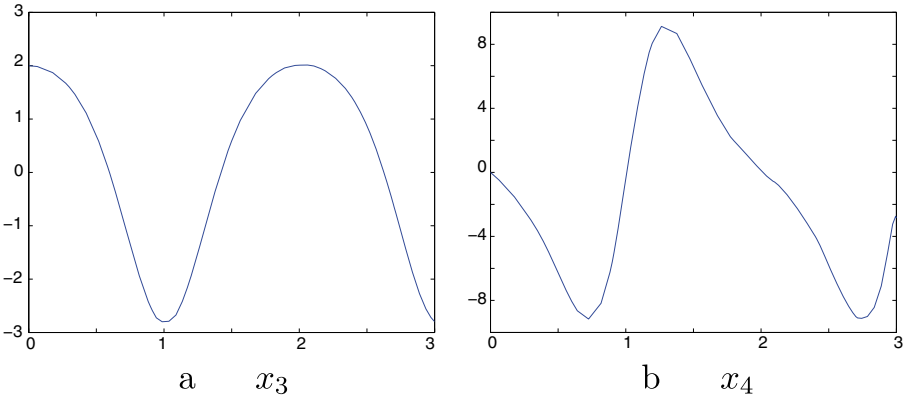


Fig. 23 Pendulum velocity variables using DTG, ($\alpha = 0$)

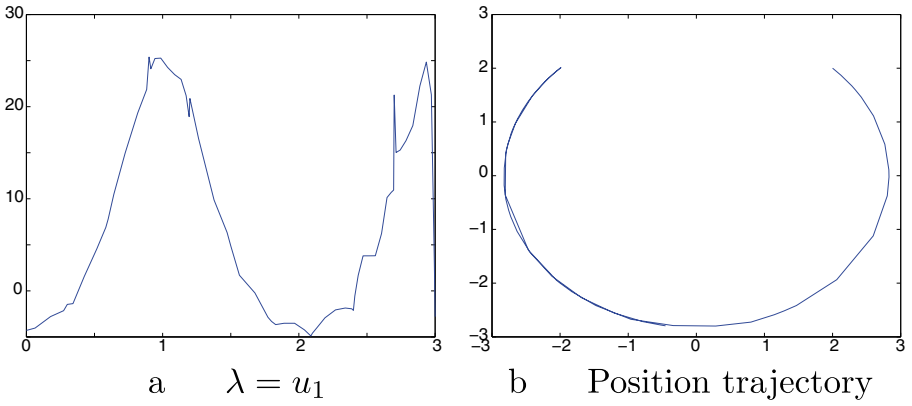


Fig. 24 High index variable and the state trajectory using DTG, ($\alpha = 0$)

However, as shown in Fig. 24 the index three state is very noisy and there is a large impulse at the end. DTG is having a much more difficult time with this index three problem that is nonlinear than it did with the linear problems solved earlier. With problems where the dynamics are nonlinear the choice of initial guess frequently becomes much more important. In fact, getting a good initial guess is sometimes the hardest part of solving the optimal control problem. We have looked at a more complex version of the pendulum problem concerning large scale motion, friction, and related effects, and both DTG and DTS were not able to solve the initial version. This leads to a long discussion of problem formulation and other related issues that requires its own somewhat lengthy paper and will not be discussed here.

Suppose now that $\alpha = 1$ and $\delta = 1$. We again have the situation mentioned earlier where when thought of as a DAE in x for a given input, we have an index three DAE, but that the virtual index is three. The results are in the following Figs. 25, 26, 27 and 28.

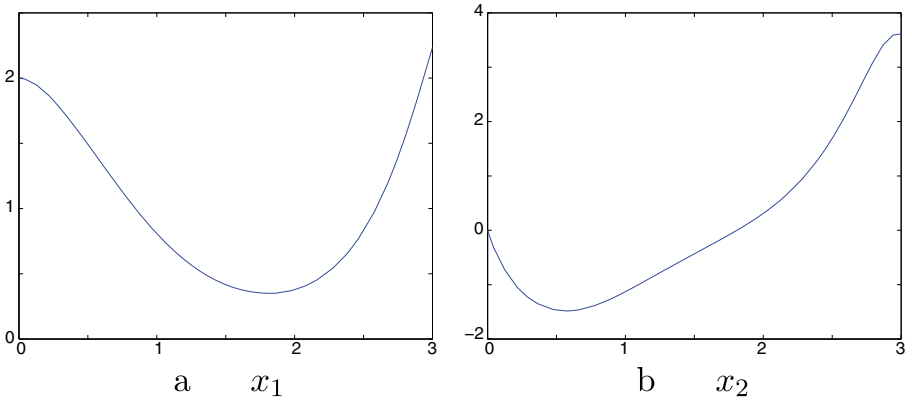


Fig. 25 Optimal positions for (20) with $\alpha = \delta = 1$ using DTG

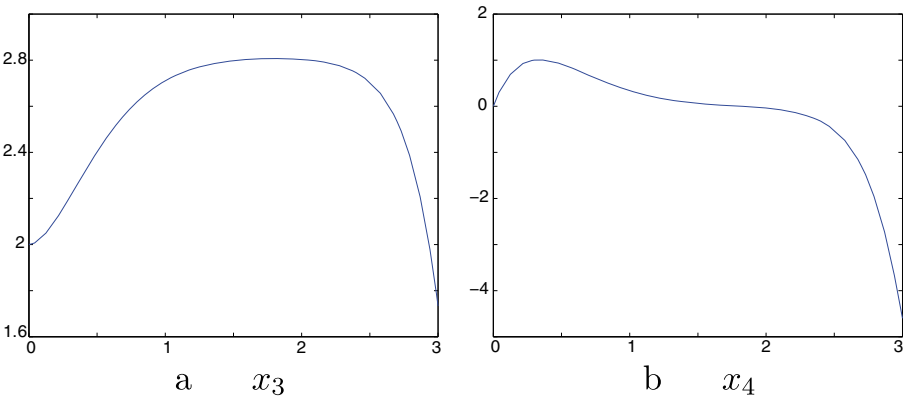


Fig. 26 Optimal velocities for (20) with $\alpha = \delta = 1$ using DTG

Looking at Figs. 25, 26 and 27, the first thing we notice is that there is no visible chatter at all and the solutions appear smooth. Because there is some control available, the solution is no longer a freely swinging pendulum. As shown in Fig. 28 the trajectory now moves along the circle but stays near the starting point.

If the presence of the control variable u creates a problem for which DTG can find a smooth solution, perhaps we can use this solution to get a good approximation of what the solution of the original problem is. That is, we take $\alpha = 1$ but δ is now large. This will penalize u and possibly keep it small, so that the resulting trajectory is close to the $\alpha = 0$ trajectory which has no u. We saw this idea was helpful with the linear Problem 9. We now examine this idea on a nonlinear problem. This computation was carried out with $\delta = 10^4$. This problem ran much faster than the index three pendulum ($\alpha = 0$) and we were able to request tighter tolerances of 10^{-5}. The computations are shown in Figs. 29, 30, 31 and 32.

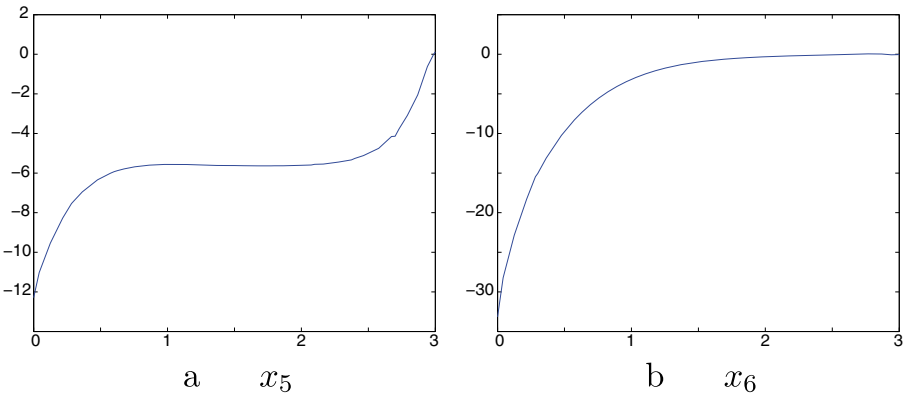


Fig. 27 Algebraic (control) variables for (20) with $\alpha = \delta = 1$ using DTG

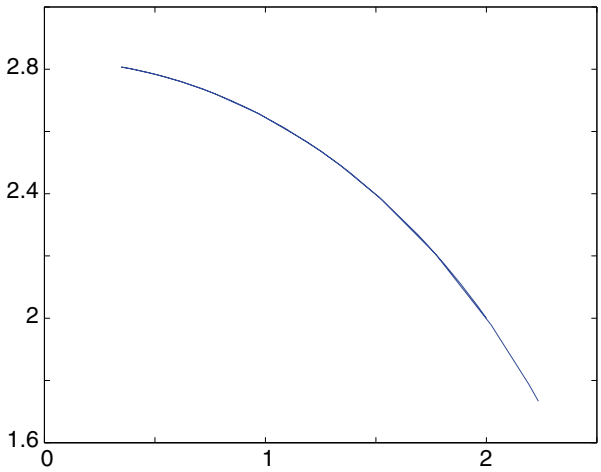


Fig. 28 Position trajectory for the optimal solution of (20) with $\alpha = \delta = 1$ using DTG

Comparing Fig. 29 to Fig. 22 we see that the regularized state trajectories in Fig. 29 are almost identical to the state trajectories in Fig. 22. Comparing Fig. 30 to Fig. 23 we see that the regularized velocity trajectories in Fig. 30 are almost identical to the velocity trajectories in Fig. 23.

Looking at Fig. 31a we see that it appears as a smoothed out version of Fig. 24a with the same amplitudes and minima. Figure 31a does have a spike at the right end as observed with some other high index problems when using DTG. There is some noise at the left in Fig. 31b. The regularized solution is smoother. As hoped for, the control u_2 is fairly small as shown in Fig. 31b. Its infinity (sup) norm is 10^{-2}. As a control u_2 gets integrated. Its definite integral on $[0,\ 3]$ is 10^{-3}.

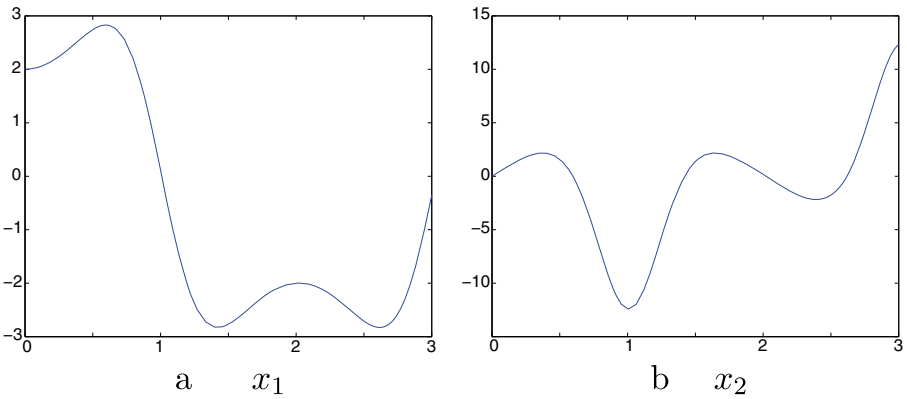


Fig. 29 Positions for problem (10) with $\alpha = 1$, $\delta = 10^4$ using DTG

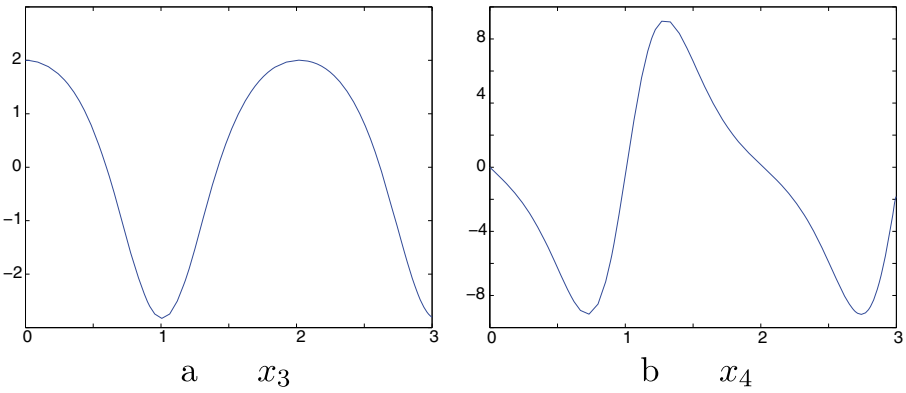


Fig. 30 Velocities for problem (10) with $\alpha = 1, \delta = 10^4$ using DTG

Figure 32 gives the state trajectory when $\delta = 10^4$ and $\alpha = 1$. Comparing this to Fig. 24b we again see the expected behavior of a swing left and then a swing back toward the right.

This example and Problem 9 illustrates some potential advantages in doing a different type of regularization by introducing a new control variable which does not lower the index and strongly weighting it in the cost. At least in this example and Problem 9 we got an improved estimate of the true solution. This idea merits further study in the future.

5.2 Using SOCX and SOCXI*

We look now at solving Problem 10 but using SOCX. This combined with our earlier computations using DTG will enable us to make a number of observations about using DT with DAEs present. In this section we include the effect of choice of

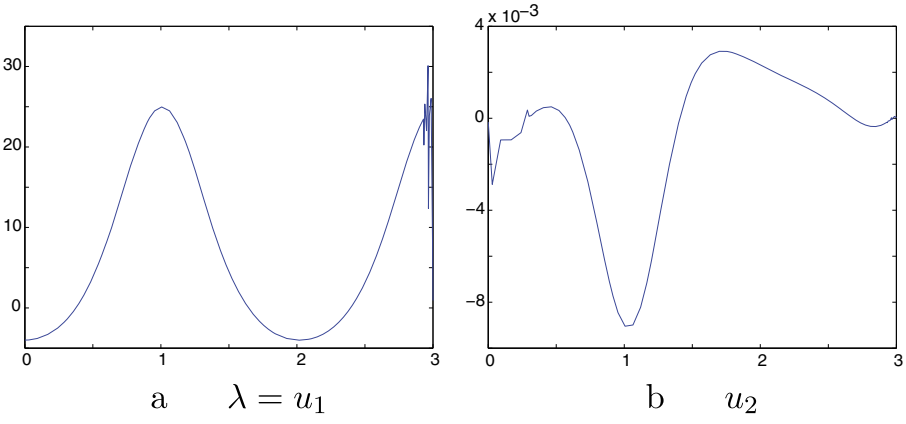


Fig. 31 High index variables for Problem 10 with $\alpha = 1, \delta = 10^4$ using DTG

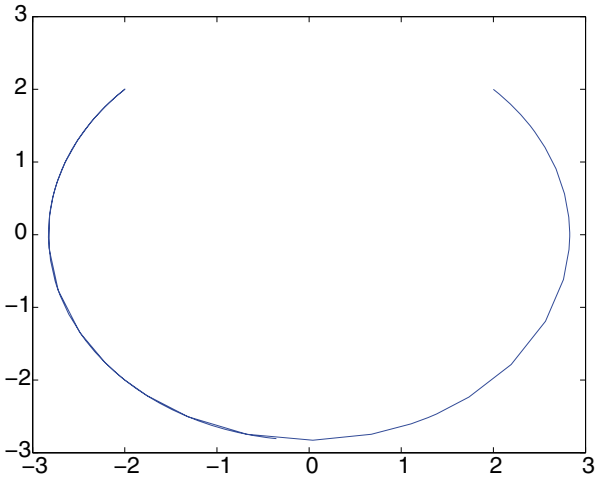


Fig. 32 State trajectory for (10) with $\delta = 10^4$ and $\alpha = 1$ using DTG

NLP optimizer. The normal procedure for SOCX is to do two iterations using the trapezoid discretization and then switch to a fourth order Hermite-Simpson method. Both methods are Lobatto IIIA IRK discretizations. SOCXI will be using the interior point optimizer and SOCXS will be using the SQP optimizer. DTG has the ability to use higher order polynomials than SOCX which could be advantageous with DAEs. There is also an experimental version of SOCX that has the option of using a four stage sixth order Lobatto method [20] instead of the two stage, fourth order Hermite-Simpson. We shall also use this discretization with an interior point method and denote it by SOCXI*.

Table 1 Summary output from solving Problem 10 using SOCXI*, SOCXS, and SOCXI

	SOCXI*			SOCXS			SOCXI		
GRID	N	ERRQ	CPU	N	ERRQ	CPU	N	ERRQ	CPU
1	10	2.4E−2	2.4E−2	10	2.4E−2	2.5E−2	10	2.4E−2	1.9E−2
2	10	6.1E−3	3.6E−2	19	6.1E−3	2.2E−2	10	6.1E−3	3.8E−2
3	19	1.1E−3	4.2E−2	19	4.8E−3	5.5E−2	19	4.8E−3	4.1E−2
4	37	2.8E−5	3.2E−2	37	2.0E−3	8.1E−2	37	2.0E−3	3.0E−2
5	73	7.4E−5	5.5E−2	69	3.9E−4	1.8E−1	69	3.9E−4	4.3E−2
6	105	1.3E−5	8.8E−2	137	5.0E−4	2.2E+0	137	5.0E−4	1.3E−1
7	186	3.3E−6	1.3E−1	189	3.6E−4	3.1E+0	189	3.6E−4	1.1E−1
8	285	3.4E−7	5.2E−1	253	3.0E+3	6.5E+0	253	2.0E−4	1.8E−1
9	486	7.5E−8	7.0E−1	505	1.0E+0	2.3E+0	355	7.3E−5	3.3E−1
	486	7.5E−8	1.6E+0	505		3.5E+0	355		

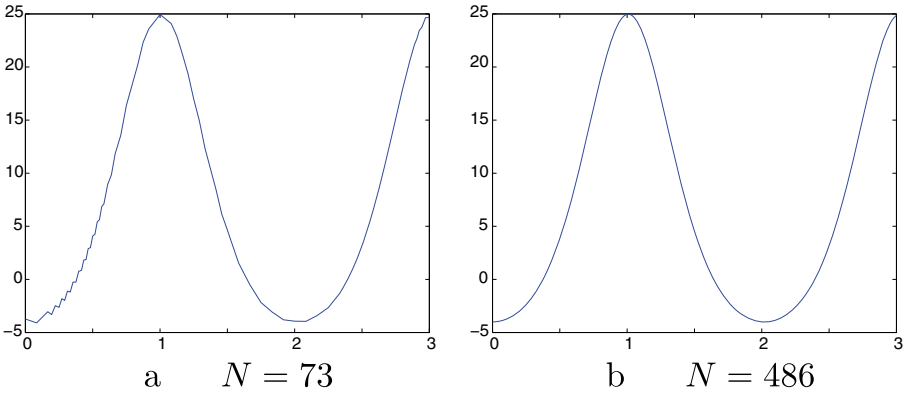


Fig. 33 High index variables for Problem 10 with $\alpha = 0$ using SOCXI*

SOCXI, SOCXS, and SOCXI* were all applied to Problem 10 with the same initial guess and a requested error tolerance of 10^{-7}. The output summaries are in Table 1. This is for the $\alpha = 0$ case so it is a nonlinear index three DAE. The columns are grid iteration number, number N of points in the grid, estimated error in the equations, and CPU time in seconds.

We shall give the solution for SOCXI* first since it was the highest quality solution. We also just give the graphs of the algebraic state variable λ since it was the highest index variable and the least smooth. By looking at several values of N we can see how the solution improves or breaks down with additional grid refinement. We can also compare the different methods.

The first thing to note from Table 1 is that SOCXI* produced a much smaller error as estimated by ERRQ. SOCXI* went to E-07 where as SOCXI appears to go to E-04 while SOCXS went to close to E-04 and then the error estimate grew larger.

We first look at SOCXI* results in Fig. 33. Visually the solutions are close from $N = 73$ to $N = 486$. From Table 1 we see that the ERRQ continued to decrease even

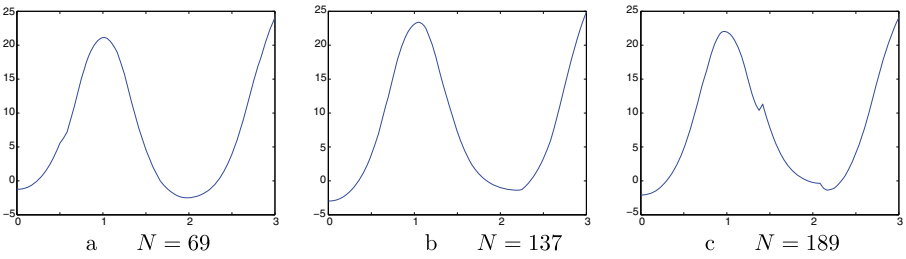


Fig. 34 High index variables for Problem 10 with $\alpha = 0$ using SOCXS

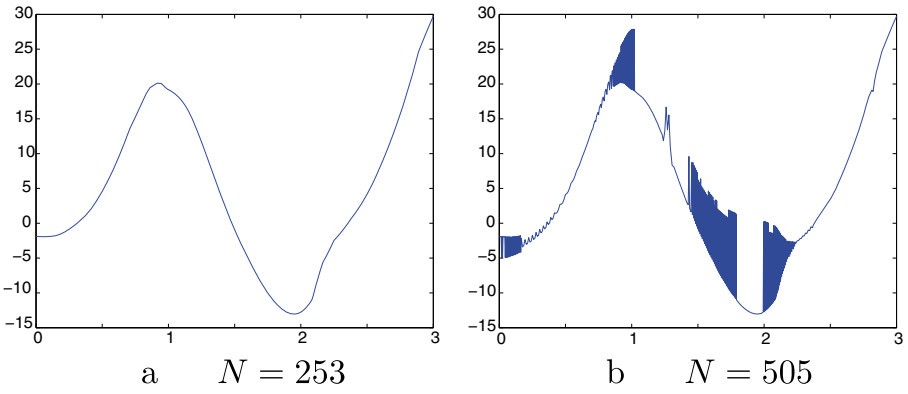


Fig. 35 High index variables for Problem 10 with $\alpha = 0$ using SOCXS

on very fine grids. A useful characteristic to note of the graphs is that the mins are essentially equal and the two maxima are essentially equal. We will use this in the comparisons and to give a quick evaluation of the other solutions.

The results using the SQP and the compressed Hermite-Simpson discretization are in Figs. 34 and 35. The results are not very satisfying. The $N = 69$ graph in Fig. 34a appears to be reasonably good. But as we refine the grid, the situation worsens. Note in Fig. 34b that the max and mins are now increasing with time. The solution quality is worse with $N = 189$ and 253 in Fig. 35. Finally in Fig. 35c we have the very noisy but wrong solution.

The SOCXI solutions are given in Figs. 36 and 37. Looking back to the SOCXI* solutions we see that there are still some perturbations but the solutions are much better behaved than the SOCXS solutions. We also do not see the breakdown if the grid is driven too fine that we see with SOCXS. However, the solution does not appear correct. Notice that in Fig. 37 that both the minimums and the maximums increase with time. This illustrates the importance of not requesting tighter tolerance than the discretization can produce on the DAEs. Smooth, but incorrect, solutions can result.

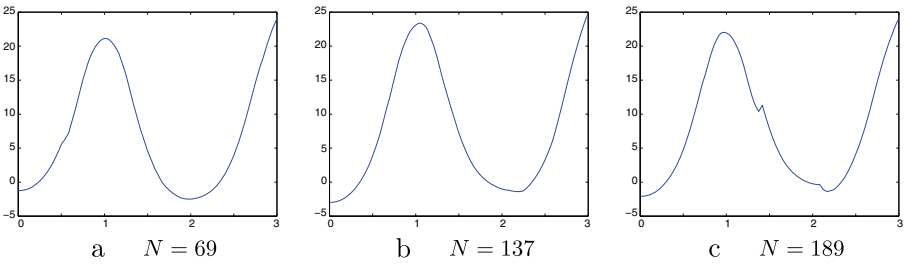


Fig. 36 High index variables for Problem 10 with $\alpha = 0$ using SOCXI

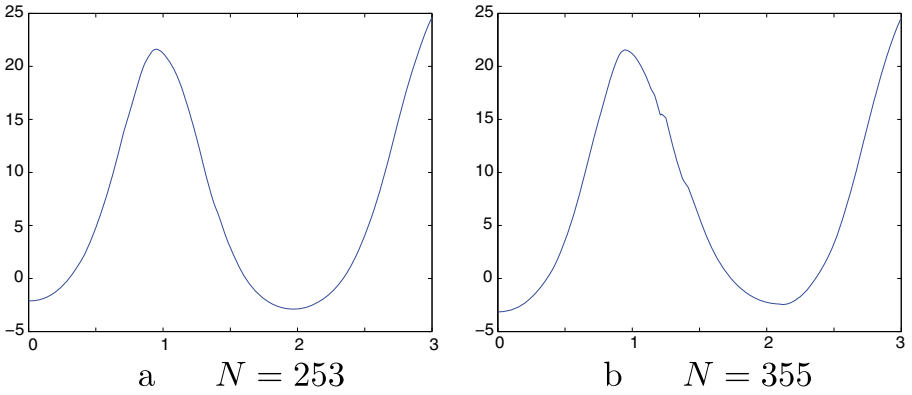


Fig. 37 High index variables for Problem 10 with $\alpha = 0$ using SOCXI

6 Conclusion

It has been shown in the literature that DT solution of constrained optimal control problems has its own theory that differs in some ways from the theory for control parameterization and some related methods. This previous work primarily used IRK discretizations (DTS). In this paper we have examined whether the results still hold for DT codes based on pseudospectral methods (DTG). It should be noted that we are not saying that a scientist or engineer should always solve constrained optimal control problems in this way. Rather we are looking at what might happen if they did solve them this way. Also we are giving some guidance on what to look for if the problem, which may have been assembled from a library of subproblems, happens to involve a higher index DAE.

It has been shown that while there are some common results, there are also some differences. When presented with a boundary value problem that is a DAE without any extra degrees of freedom, DTG does better at solving index two and three DAEs than DTS. DTG was able to solve a number of linear and nonlinear problems with some reduction of order and some chatter at the right endpoint in the highest index variables. This chattering may be due to the particular implementation of the pseudospectral method in GPOPS II which results in a free control on the endpoint which is then effected by the cost function. We have pointed out how to remove this chatter from the variable of interest at the cost of a lower order solution from a higher index problem.

Both DTS and DTG can exploit the fact that what appears to be a higher index optimal control problem has virtual index one. However, DTG was able to get useful solutions for some optimal control problems that did not have virtual index one. It was shown that this idea could be used to regularize some higher index problems in the sense that the index was not reduced but a smoother approximation of the solution was found and it was found more quickly by modifying the cost function. The better behavior with higher index DAEs is likely due to GPOPS use of higher order polynomials. This is supported by the success of SOCXI*.

Both DTS and DTG are able to solve some inequality constrained problems where for the optimal solution the inequality constraint is an equality on an arc resulting in an higher index DAE. However, the mechanism appears to be somewhat different. DTS can do small perturbations off the constraint and use these to cancel the unstable error growth. With DTG it is not clear if there is a similar mechanism or if DTG just does a better job with some index two or three DAEs.

Both DTS and DTG are able to regularize a problem where the necessary equations would say there are touch points existing on small time and space scales.

DTS is well understood. The results in this paper show that DTG has some interesting properties when working with DAE optimal control problems but the mechanisms and the results are not identical with those of DTS and this needs to be examined more carefully.

In our examination of DT and higher index DAEs we also looked at an index 3 nonlinear pendulum problem. The usefulness of higher order polynomials when working with higher index DAEs was clearly seen. The best performance was by SOCXI* which used a sixth order Lobatto IRK method. The addition of this sixth order method appears to add a promising capability to SOCX at least for the kinds of problems discussed here.

One lesson from these calculations is that if a user is going to use DT with higher index DAEs, then the user should not blindly trust the final solution. It is possible to start to get an approximation and then do worse as the grid is refined as shown by SOCXS. In fact, the best solution of those plotted was at N=69. It is possible to get a reasonable appearing answer but to have some drift as shown by SOCXI where the apparent convergence in Fig. 36 was not to the correct solution.

The use of interior point methods seemed to perform more robustly than SQP methods at least on these examples supporting observations made by others on other types of singular problems.

Acknowledgments The helpful comments of reviewers of an earlier version of this paper were much appreciated and have resulted in an improved paper.

References

1. Balla, K., Kurina, G.A., März, R.: Index criteria for differential algebraic equations arising from linear-quadratic optimal control problems. J. Dyn. Control. Syst. **12**, 289–311 (2006)
2. Betts, J.T.: Practical methods for optimal control and estimation using nonlinear programming, 2nd ed. SIAM, Philadelphia (2010)
3. Betts, J.T., Biehn, N., Campbell, S.L.: Convergence of nonconvergent IRK discretizations of optimal control problems with state inequality constraints. SIAM J. Sci. Comp. **23**, 1981–2007 (2002)
4. Betts, J.T., Campbell, S.L., Engelsone, A.: Direct transcription solution of optimal control problems with higher order state constraints: theory vs practice. Optim. Eng. **8**, 1–19 (2007)
5. Betts, J.T., Campbell, S.L., Engelsone, A.: Direct transcription solution of inequality constrained optimal control problems, pp. 1622–1626. Proceedings of the American Control Conference, Boston (2004)
6. Betts, J.T., Erb, S.O.: Optimal low thrust trajectories to the Moon. SIAM J. Appl. Dyn. Syst. **2**, 144–170 (2003)
7. Biegler, L.T., Campbell, S.L., Mehrmann, V. (eds.): Control and optimization with differential-algebraic constraints. SIAM, Philadelphia (2012)

8. Biegler, L.T.: Nonlinear programming strategies for dynamic chemical process optimization. Theor. Found. Chem. Eng. **48**, 541–554 (2014)
9. Brenan, K.E., Campbell, S.L., Petzold, L.R.: Numerical solution of initial-value problems in differential-algebraic equations. SIAM, Philadelphia (1996)
10. Campbell, S.L., März, R.: Direct transcription solution of high index optimal control problems and regular Euler-Lagrange equations. J. Comput. Appl. Math. **202**, 186–202 (2007)
11. Campbell, S.L., Kunkel, P., Mehrmann, V.: Regularization of linear and nonlinear descriptor systems. In: Biegler, L.T., Campbell, S.L., Mehrmann, V. (eds.) Control and optimization with differential-algebraic constraints, pp. 17–34. SIAM, Philadelphia (2012)
12. Darby, C.L., Hager, W.W., Rao, A.V.: An hp-adaptive pseudospectral method for solving optimal control problems. Optimal Control Applications and Methods **32**, 476–502 (2011)
13. Engelsone, A., Campbell, S.L., Betts, J.T.: Direct transcription solution of higher-index optimal control problems and the virtual index. Appl. Numer. Math. **57**, 281–296 (2007)
14. Engelsone, A., Campbell, S.L., Betts, J.T.: Order of Convergence in the direct transcription solution of optimal control problems, pp. 3723–3728. Proceedings of the IEEE Conference on Decision Control - European Control Conference, Seville (2005)
15. Ghanbari, F., Goreishi, F.: Convergence analysis of the pseudospectral method for linear DAEs of index-2. Int. J. Comp. Methods **10**, 1350019-1–1350019-20 (2013)
16. Hosseini, M.M.: Pseudospectral method for numerical solution of DAEs with an error estimation. Appl. Math. Comput. **170**, 115–124 (2005)
17. Hosseini, M.M.: A domain decomposition method for solution of nonlinear differential algebraic equations. Appl. Math. Comput. **181**, 1737–1744 (2006)
18. Hosseini, M.M.: Numerical solution of linear high-index DAEs, Computational Science and Its Applications ICCSA 2004, vol. 305, pp. 676–685. Lecture Notes in Computer Science, Springer (2004)
19. Jacobsen, D.H., Lele, M.M., Speyer, J.L.: New necessary conditions of optimality for control problems with state variable inequality constraints. J. Math. Anal. Appl. **35**, 255–284 (1971)
20. Jay, L.O.: Lobatto methods. In: Engquist, B. (ed.) Encyclopedia of applied and computational mathematics, numerical analysis of ordinary differential equations. Springer - The Language of Science (2013)
21. Kameswaran, S., Biegler, L.T.: Convergence rates for direct transcription of optimal control problems using collocation at Radau points. Computat. Optim. Appls. **41**, 81–126 (2008)
22. Kameswaran, S., Biegler, L.T.: Simultaneous dynamic optimization strategies: recent advances and challenges. Comput. Chem. Eng. **30**, 1560–1573 (2006)
23. Kameswaran, S., Biegler, L.T.: Advantages of nonlinear-programming-based methodologies for inequality path-constrained optimal control problems - a numerical study. SIAM J. Sci. Stat. Comput. **30**, 957–981 (2008)
24. Kunkel, P., Mehrmann, V.: Differential-algebraic equations: analysis and numerical solution. European Mathematical Society, Zürich (2006)
25. Lamour, R., März, R., Tischendorf, C.: Differential-algebraic equations: a projector based analysis. Differential Algebraic Equations Forum, Springer (2012)
26. März, R.: Notes on linearization of DAEs and on optimization with differential-algebraic constraints. In: Biegler, L.T., Campbell, S.L., Mehrmann, V. (eds.) Control and optimization with differential-algebraic constraints, pp. 37–58. SIAM, Philadelphia (2012)
27. Patterson, M.A., Rao, A.V.: GPOPS II: A MATLAB software for solving multiple-phase optimal control problems using hp-adaptive Gaussian quadrature collocation methods and sparse nonlinear programming. ACM Trans. Math Software **41**, 1–37 (2014)
28. Rao, A.V., Benson, D.A., Darby, C., Patterson, M.A., Francolin, C., Sanders, I., Huntington, G.T.: Algorithm 902: Gpops, a MATLAB software for solving multiple-phase optimal control problems using the Gauss pseudospectral method. ACM Trans. Math. Softw. **37**, 22:1–22:39 (2010)
29. Saravi, M., Babolian, E., England, R., Bromilow, M.: System of linear ordinary differential and differential-algebraic equations and pseudo-spectral method. Computers and Mathematics with Applications **59**, 1524–1531 (2010)
30. Wächter, A., Biegler, L.T.: On the Implementation of a primal-dual interior point filter line search algorithm for large-scale nonlinear programming. Math. Program. **106**, 25–57 (2006)
31. Yaganeh, S., Saadatmandi, A., Soltanian, F., Dehghan, M.: The numerical solution of differential-algebraic equations by sinc-collocation method. Comp. Appl. Math. **32**, 343–354 (2013)